

Rapport Technique – Infrastructure IAM / SSO / SIEM

Résumé

Ce projet documente la conception, la mise en œuvre et l'évaluation sécuritaire d'une infrastructure IAM complète, construite en environnement de lab personnel pour simuler un déploiement entreprise réel.

Le projet s'articule autour de 5 axes :

- La mise en place d'un SSO centralisé via Keycloak fédéré sur un AD Windows Server
- L'intégration sur Grafana et Nextcloud avec une gestion des droits basés sur les utilisateurs de l'AD
- La démonstration d'une attaque MITM (Man In The Middle) sur le trafic LDAP en clair pour interception des credentials et énumération complète de l'AD
- Le hardening de l'infrastructure via passage en LDAPS pour chiffrer les échanges et rendre l'interception inutile
- La supervision des événements via l'implémentation d'un SIEM Wazuh en configurant des règles de détection personnalisées à l'environnement critique

Résultat : L'infrastructure est opérationnelle, fonctionne normalement et est sécurisée. Le SSO fonctionne pour les deux applications métier Grafana et Nextcloud. Wazuh supervise la sécurité des événements de Keycloak via des règles et dashboard définis. L'attaque émise en claire visant à intercepter le LDAP a été réalisé avec succès, montrant la vulnérabilité critique du non chiffrement, nécessitant le LDAPS.

Analyse Risques / Opportunités

Opportunités :

- Centralisation des accès via un point central d'administration (permettant le blocage d'un compte sur l'AD en cas de compte compromis, l'attaquant sera complètement bloqué)
- Gestion des droits par rôles RBAC (rôles définis dans Keycloak et propagés via des tokens sans gestion des droits sur les applications)
- SSO (une seule authentification donnant accès à toutes les applications)
- Réduction de la surface d'attaque et traçabilité centralisée pour détection via SIEM
- Scalabilité (une nouvelle appli au SSO ne nécessite pas la création de comptes utilisateurs car les politiques et droits seront appliqués automatiquement)

Risques :

- Utiliser le protocole LDAP en clair (capturable via un simple tcpdump)
- Keycloak vulnérable comme SPOF (Single Point Of Failure, point unique de défaillance en cas de compromission, besoin de sécuriser davantage ce service)
- SIEM aveugle au sniffing (= écouter le trafic, besoin d'un IDS)

Environnement lab

- Serveur Docker en Ubuntu 24 : Keycloak, Nextcloud, Grafana et Wazuh
 - Keycloak : Realm : tp-local, User Federation -> AD LDAPS, MFA activé, Clients OIDC : nextcloud et Grafana, Utilisateurs : jdeloin / mansse / niamst, Rôles realm : grafana-admin/editor/viewer et nextcloud-user/admin
 - Nextcloud : dossiers de partage : RH, Communication et Marketing
 - Grafana: rôles -> jdeloin Admin / mansse Editor / niamst Viewer
- Serveur Windows 2022: AD et AD CS (PKI)
 - 7 comptes AD réparti dans les groupes métiers GRP_RH, GRP_Communication, GRP_Marketing
- Client Windows : tests et accès aux services
- Kali : tests d'intrusion

Composant	Adresse IP / Port	Rôle
Active Directory	192.168.1.10 — Port 389/636	Annuaire LDAP/LDAPS
Keycloak	192.168.1.124 — Port 8080	IdP SSO / Fédération LDAP
Grafana	192.168.1.124 — Port 3000	Tableau de bord — SSO
Nextcloud	192.168.1.124 — Port 80/443	Partage de fichiers — SSO
Wazuh	192.168.1.124 — Port 1514/1515 pour le Manager et 443 pour le Dashboard	SIEM — Collecte & analyse
Machine attaquante	Réseau local	Tests de pénétration

Étape 1 – Déploiement de Keycloak

Keycloak est déployé via Docker sur le serveur ubuntu avec persistance de la base de données via un volume bindmount puisque sans ce volume toute la configuration sera perdue à l'arrêt du conteneur. Le realm tp-local est créé, l'AD est connecté en LDAP. Un realm tp-local (entité) est créé dans Keycloak pour ce projet.

Étape 2 – Connexion AD via LDAP

Dans le realm créé, on configure la connexion LDAP dans User Federation en mode READ_ONLY pour synchroniser les utilisateurs et groupes AD. On renseigne l'URL de connexion, le DN et les identifiants du compte admin récupéré via powershell :

Get-ADUser -Identity Administrateur | Select DistinguishedName

Ainsi que les attributs LDAP spécifiques à AD (sAMAccountName, objectGUID, Search scope Subtree). Une fois la connexion testée et validée, la synchronisation se fait manuellement.

The screenshot shows the Keycloak administration interface for the 'tp-local' realm. The left sidebar contains navigation links: Manage, Clients, Client scopes, Realm roles, Users, Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, and User federation. The 'User federation' section is active, showing the configuration for an LDAP provider. The 'Bind DN' field is set to 'CN=Administrateur,CN=Users,DC=tp-local,DC=fr'. The 'Bind credentials' field is masked with dots. A 'Test authentication' button is present. Under the 'LDAP searching and updating' section, the 'Edit mode' is set to 'READ_ONLY'. The 'Users DN' field is 'CN=Users,DC=tp-local,DC=fr'. The 'Username LDAP attribute' is 'sAMAccountName'. The 'RDN LDAP attribute' is 'cn'. The 'UUID LDAP attribute' is 'objectGUID'. The 'User object classes' are 'person, organizationalPerson, user'.

Étape 3 – Déploiement Nextcloud avec HTTPS

Nextcloud est déployé en docker derrière un reverse proxy avec un certificat auto-signé OpenSSL car le plugin OIDC Nextcloud exige HTTPS.

Problème rencontré : Nextcloud refusait les connexions OIDC avec "Vous devez accéder à Nextcloud via HTTPS" malgré l'accès HTTPS. La commande `allow_local_remote_servers` est indispensable pour autoriser Nextcloud à contacter des IPs privées (ici Keycloak sur 192.168.1.124).

Étape 4 – Intégration SSO Nextcloud- Keycloak

Création du client OIDC Nextcloud dans Keycloak avec l'IP de redirection en pointant vers le Discovery Endpoint de Keycloak. Les utilisateurs AD peuvent désormais se connecter à Nextcloud via le bouton « Se connecter avec Keycloak ».

Problème rencontré : "The discovery endpoint is not reachable" malgré une URL correcte — résolu par `allow_local_remote_servers`.



Étape 5 – Synchronisation des groupes AD vers Nextcloud

3 groupes sont créés dans l'AD (GRP_RH, GRP_Communication, GRP_Marketing) et synchronisés dans Keycloak via un mapper group-ldap-mapper.

Problème rencontré : La synchronisation échouait car les groupes système AD se référencent mutuellement et Keycloak ne trouvait pas tous les membres. Résolu en activant l'option Ignore Missing Groups dans le mapper.

Résultat : 23 groupes synchronisés (3 groupes métier + groupes système AD).

Un second mapper Group Membership est ajouté au client nextcloud pour inclure les groupes dans le token JWT. Dans Nextcloud, l'application Team Folders est installée pour créer des dossiers partagés accessibles automatiquement à tout membre d'un groupe dès sa première connexion (contrairement aux partages classiques qui ne s'appliquent pas aux nouveaux comptes OIDC créés après la création du partage)

Étape 6 – MFA via Keycloak (TOTP)

Le MFA est activé dans Keycloak dans Authentication -> Flows -> browser -> Browser – Conditional OTP -> Required. Il s'applique automatiquement à toutes les applications connectées à Keycloak sans les configurer une à une.

Étape 7 – Déploiement Grafana et intégration OIDC avec les rôles

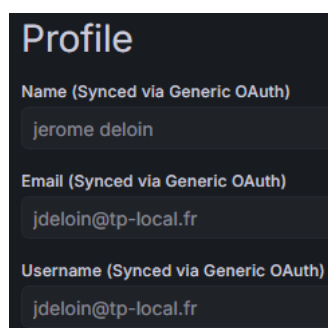
Grafana est intégré à Keycloak via la création d'un client et 3 realm rôles sont créés dans Keycloak : grafana-admin, grafana-editor, grafana-viewer. Un mapper User Realm Role est également ajouté au client pour transmettre les rôles dans le token. Grafana est intégré à Keycloak via Generic OAuth (OIDC). La particularité ici est que toute la configuration des rôles se fait directement dans le docker-compose.yml via des variables d'environnement.

```
GF_AUTH_GENERIC_OAUTH_ROLE_ATTRIBUTE_PATH: "contains(roles[*], 'grafana-admin')
&& 'Admin' || contains(roles[*], 'grafana-editor') && 'Editor' || 'Viewer'"
```

Trois utilisateurs ont donc des rôles différents :

- jdeloin : grafana-admin – rôle admin
- mansse : grafana-editor – rôle editor
- niamst : grafana-viewer – rôle viewer

Problème rencontré : Tentative d'intégration via SAML — fonctionnalité Enterprise (payante) dans Grafana. Résolu en utilisant Generic OAuth (OIDC), disponible dans la version open source.



Organizations	
Name	Role
Main Org.	Admin

Profile

Name (Synced via Generic OAuth)

mathieu ansse

Email (Synced via Generic OAuth)

mansse@tp-local.fr

Username (Synced via Generic OAuth)

mansse@tp-local.fr

Organizations	
Name	Role
Main Org.	Editor

Étape 8 – Utilisateurs locaux Keycloak et contrôle d'accès par rôles sur Nextcloud

Cette partie démontre la cohabitation d'utilisateurs AD et d'utilisateurs locaux Keycloak, avec une politique d'accès différenciée par application.

Utilisateur	Rôle Grafana	Rôle Nextcloud
jdeloin	Admin	Utilisateur
mansse	Editor	Administrateur
niamst	Viewer	Aucun accès (bloqué)

Problème rencontré : En mode WRITABLE sur User Federation, Keycloak tentait d'écrire dans l'AD et échouait. Il faut créer les utilisateurs locaux uniquement en mode READ_ONLY pour qu'ils soient stockés localement dans Keycloak.

Problème rencontré : Le mapper Group Membership (groupes AD) et le mapper User Realm Role (rôles applicatifs) écrivaient tous les deux dans le claim groups. Solution : changer le Token Claim Name du mapper de rôles de groupes vers roles, et passer le scope global Keycloak roles de Default à Optional sur le client nextcloud pour éviter le doublon.

Le token JWT résultant pour jdeloin est structuré comme ci-dessous :

```

json
{
  "groups": ["GRP_Marketing", ...],
  "roles": ["nextcloud-user", "grafana-admin"]
}

```

Dans Nextcloud, la restriction d'accès est configurée via une whitelist regex nextcloud-user|nextcloud-admin, ainsi tout utilisateur sans l'un de ces rôles est bloqué à la connexion.

Problème rencontré : Le plugin user_oidc ne mappe pas automatiquement les groupes OIDC vers le groupe admin natif Nextcloud. On doit les attribuer en lignes de commandes via occ.

```
docker exec -u www-data nextcloud php occ user:list
```

```
docker exec -u www-data nextcloud php occ group:adduser admin <ID_HASH_UTILISATEUR>
```

+ Nouveau compte		Nom d'affichage	E-mail	Groupes
Tous les comptes	14	MA mathieu ansse	mansse@tp-local.fr	nextcloud-admin, admin
Administrateurs	2	A admin		admin
Récemment actifs	14			

Étape 9 – Exploitation de la vulnérabilité LDAP en clair

Le scénario d'attaque est réaliste, demande peu de ressources et nécessite seulement un scan basique. Un attaquant avec Kali écoute localement le trafic LDAP généré par Keycloak vers l'AD, sans aucun mouvement réseau, ce qu'un IDS pourrait détecter. La capture est effectuée depuis Kali via tcpdump en ciblant le port 389 (LDAP) et l'IP de l'AD :

```
sudo tcpdump -i any host 192.168.1.10 and port 389 -w /tmp/ldap_capture.pcap
```

La capture attend qu'une connexion SSO soit provoquée sur Nextcloud par exemple avec un compte utilisateur ou admin. Keycloak envoie une requête LDAP bindRequest à l'AD pour vérifier les identifiants. Ensuite le fichier de capture est analysé dans un outil type Wireshark avec le filtre ldap et les paquets bindRequest révèlent le Distinguished Name (DN) du compte avec les identifiants en clair.

Ensuite, avec les identifiants récupérés il est possible d'énumérer l'AD avec les noms, mails, groupes et attributs de compte. Cela démontre l'impact réel d'une interception LDAP :

```
ldapsearch -x -H ldap://192.168.1.10 -D "CN=jdeloin,DC=tp-local,DC=fr" -w "mdp" \
-b "DC=tp-local,DC=fr" "(objectClass=user)"
```

```
(root@kali)-[/home/edugelay/Projet_Wazuh]
# cat export_ad
dn: CN=Administrateur,CN=Users,DC=tp-local,DC=fr
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: user
cn: Administrateur
description:: Q29tcHRlIGTigJl1dGlsaXNhGV1ciBk4oCZYWRtaW5pc3RyYXRpb24=
distinguishedName: CN=Administrateur,CN=Users,DC=tp-local,DC=fr
instanceType: 4
whenCreated: 20250709174326.0Z
whenChanged: 20260514123807.0Z
uSNCreated: 8196
memberOf:: Q049UHJvcHJpw6l0YWlyZXMGY3LDQWFOZXVycyBkZSBsYSBzdHJhdMopZ2llIGRlIGd
yb3VwZSxDTj1Vc2VycyxEQz10cC1sb2NhbCxEQz1mcg==
memberOf: CN=Admins du domaine,CN=Users,DC=tp-local,DC=fr
memberOf:: Q049QWRtaW5pc3RyYXRldXJzIGRlIGZigJlbnRyZXByaXNlLENOPVZzZXJzLERDPXR
wLWxvY2FsLERDPWZy
memberOf:: Q049QWRtaW5pc3RyYXRldXJzIGR1IHNjaMOpbWESQ049VXNlcNMsREM9dHAtbG9jYWw
sREM9ZnI=
memberOf: CN=Administrateurs,CN=Builtin,DC=tp-local,DC=fr
```

Étape 10 – Sécurisation LDAP-> LDAPS

La mesure de sécurité pour contourner l'attaque précédente est de chiffrer les requêtes transmises. Ainsi, le protocole LDAPS doit être configuré via l'AD Certificate Services qui est installé comme Root CA Entreprise, ce qui va déclencher automatiquement la génération d'un certificat LDAPS sur le port 636.

LDAPS est la réponse technique directe à cette vulnérabilité. Le chiffrement TLS rend toute interception inutile même pour un attaquant positionné sur le serveur Keycloak. Les solutions complémentaires sont la segmentation réseau pour limiter le port 389 aux seules IPs autorisées, un IDS réseau (Suricata, Snort) pour l'analyse du trafic en temps réel, et l'agent Wazuh sur le Windows Server pour remonter les événements AD.

La connexion LDAPS est vérifiée depuis le serveur ubuntu via openssl s_client :

```
openssl s_client -connect 192.168.1.10:636 -showcerts
```

Ensuite, le certificat CA est exporté depuis le serveur windows sur le serveur ubuntu. La connexion dans Keycloak est mise à jour en utilisant désormais ldaps://192.168.1.10:636

Étape 11 – Déploiement Wazuh (SIEM)

Afin de veiller à la traçabilité, un SIEM Wazuh est déployé en docker que l'on relie après installation au serveur ubuntu via un agent.

Ensuite, les logs Keycloak sont encapsulées dans un JSON Docker sur le serveur. Wazuh ne parse pas nativement ce double format alors on redirige les logs vers un fichier brut sur le serveur ubuntu

```
docker logs -f keycloak 2>&1 | grep "keycloak" >> /var/log/keycloak.log &
```

Les événements LOGIN_ERROR de Keycloak apparaissent dans Wazuh Discover avec un rule.id : 100010 et un rule.groups : keycloak.

Time	_source
May 14, 2026 @ 14:38:09.525	<pre>predecoder.timestamp: 2026-05-14 09:01:23.371 input.type: log agent.ip: 192.168.1.124 agent.name: Ubuntu_server agent.id: 001 manager.name: wazuh.manager rule.firedtimes: 6 rule.mail: false rule.level: 5 rule.description: Keycloak: Echec de connexion rule.groups: keycloak rule.id: 100010 location: /var/log/keycloak.log decoder.parent: keycloak decoder.name: keycloak id: 1778762289.55212 full_log: 2026-05-14 09:01:23.371 WARN [org.keycloak.events] (executor-thread-17) type="LOGIN_ERROR", realmId="17a4b0f4-713a-496e-9afa-819bd94085ab", clientId="security-admin-console", userId="b2788572-48b7-49e6-8896-aa5cb670ad07", ipAddress="192.168.1.158", error="invalid_user_credenti</pre>

Pour améliorer efficacement l'utilisation de Wazuh, un système d'alerte peut-être défini et relié à un diagramme nous permettant de suivre les échecs de connexion via le SSO.



Ce diagramme est relié à des filtres de détection en fonction des besoins. Par exemple, le filtre « Alertes critiques » pointe sur `rule.level >= 7`, où sur Wazuh le niveau de règle est variable de 0 à 15. Lorsque le seuil d’alerte est égal ou supérieur au niveau 7, Wazuh la considère comme une alerte notable, nous permettant d’alimenter le diagramme vu.

Saved Queries
Save query text and filters that you want to use again.
[Alertes critiques](#)
[Echec de connexion](#)
[Keycloak - All events](#)
< 1 >

Search
[rule.level >= 7](#)
[Echec de connexion](#)
[rule.groups: keycloak](#)
[Keycloak: Echec de connexion](#)
[rule.id: 100010](#)

Ainsi, l’infrastructure est désormais opérationnelle et supervisée via le SIEM.